**Experiment No.:** 4

**Experiment Name:** Aggregate Functions and Grouping Operations

**Aim:** To perform SQL queries using aggregate functions and grouping operations.

## PART B

## (PART B: TO BE COMPLETED BY STUDENTS)

**Students must submit the soft copy as per the following segments within two hours of the practical. The soft copy must be uploaded on Blackboard or emailed to the concerned lab-in-charge faculty at the end of the practical in case there is no Blackboard access available.**

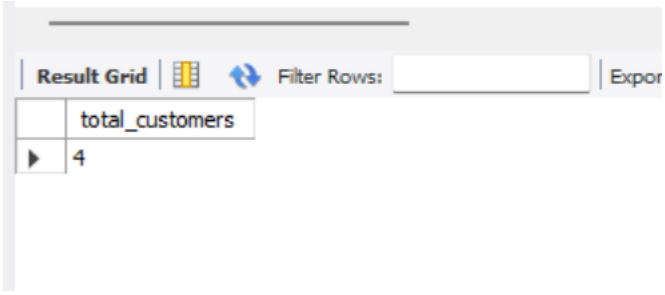
| | |
|---|---|
| Roll. No. C-60 | Name: Avdhut Madhavi |
| Class: C | Batch: C2 |
| Date of Experiment | Date of Submission |
| Grade | |

### B.1: Practical Questions

Frame Questions based on your case study.Execute the statement and check the output. Put the SQL query and its **output after each question.**

**1. Count the total number of customers**

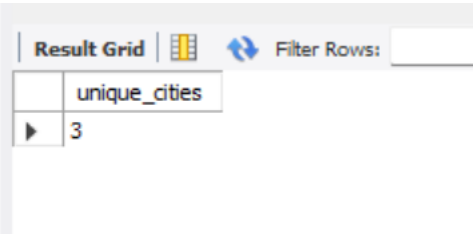
```
SELECT COUNT(*) AS total_customers
FROM customer;
```




**Explanation:** Counts all rows in the customer table.

**2. Count unique customer cities**

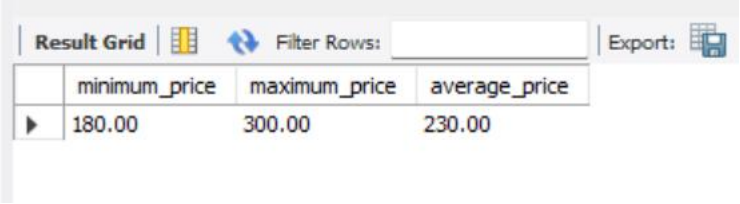
```
SELECT COUNT(DISTINCT city) AS unique_cities
FROM customer;
```




**Explanation:** Counts the number of distinct, non-NULL cities.

### 3. Find the minimum, maximum, and average ticket price

```
SELECT
   MIN(ticket_price) AS minimum_price,
   MAX(ticket_price) AS maximum_price,
   ROUND(AVG(ticket_price), 2) AS average_price
FROM showtime;
```

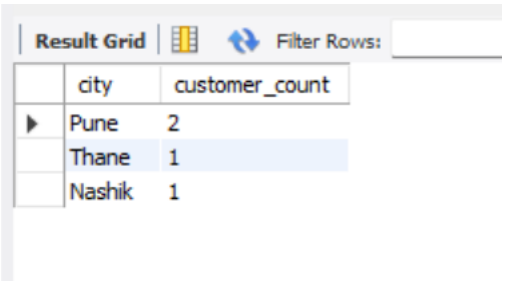



| minimum_price | maximum_price | average_price |
|---|---|---|
| 180.00 | 300.00 | 230.00 |

**Explanation:** Calculates the lowest, highest, and average ticket prices.

### 4. Count customers by city

```
SELECT city, COUNT(*) AS customer_count
FROM customer
GROUP BY city;
```

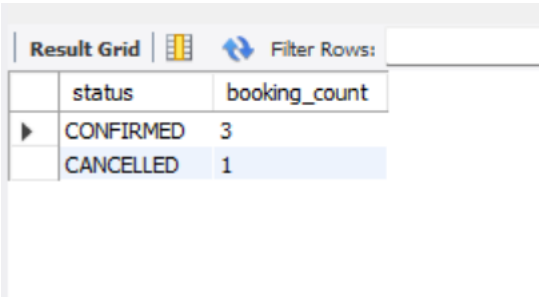



| city | customer_count |
|---|---|
| Pune | 2 |
| Thane | 1 |
| Nashik | 1 |

**Explanation**: Groups customers by city and counts the customers in each group.

### 5. Count bookings by status

```
SELECT status, COUNT(*) AS booking_count
FROM booking
GROUP BY status;
```




| status | booking_count |
|---|---|
| CONFIRMED | 3 |
| CANCELLED | 1 |

**Explanation:** Counts bookings with each status, such as CONFIRMED and CANCELLED.

### 6. Count bookings for each show

```
SELECT show_id, COUNT(*) AS total_bookings
FROM booking
GROUP BY show_id
ORDER BY show_id;
```

Result Grid | Filter Rows:

| show_id | total_bookings |
|---|---|
| 401 | 1 |
| 402 | 1 |
| 403 | 1 |
| 404 | 1 |

**Explanation:** Groups bookings by show ID and counts the bookings for each show.

**7. Display booking statuses having more than one booking**

```
SELECT status, COUNT(*) AS booking_count
FROM booking
GROUP BY status
HAVING COUNT(*) > 1;
```

Result Grid | Filter Rows:

| status | booking_count |
|---|---|
| CONFIRMED | 3 |

**Explanation:** Displays only booking-status groups containing more than one record.

**8. Display show IDs whose average payment amount is greater than ₹200**

```
SELECT b.show_id,
      ROUND(AVG(p.amount), 2) AS average_payment
FROM booking b
JOIN payment p
   ON b.booking_id = p.booking_id
GROUP BY b.show_id
HAVING AVG(p.amount) > 200;
```

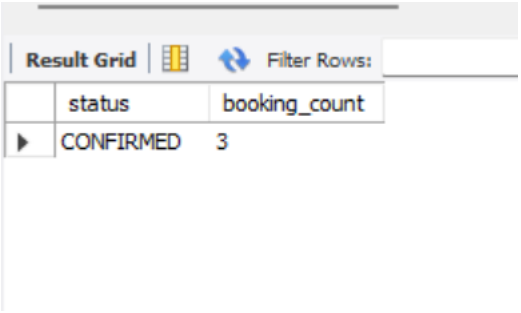
Result Grid | Filter Rows:

| show_id | average_payment |
|---|---|
| 402 | 250.00 |
| 404 | 220.00 |

**Explanation:** Calculates the average payment for each show and displays only shows with an average payment above ₹200.

**9. Count bookings by status for confirmed bookings only**

```
SELECT status, COUNT(*) AS booking_count
FROM booking
WHERE status = 'CONFIRMED'
GROUP BY status;
```



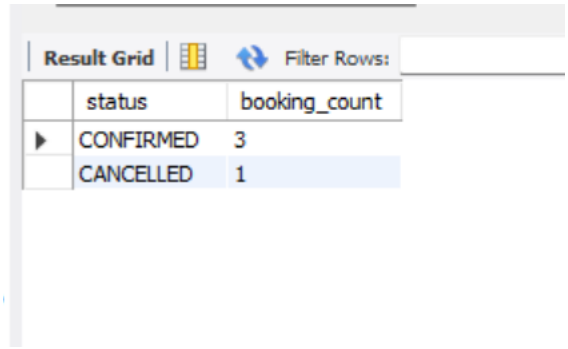
Result Grid | Filter Rows:

| status | booking_count |
|---|---|
| CONFIRMED | 3 |

**Explanation:** Filters confirmed bookings first and then counts them by status.

**10. Sort booking statuses by the number of bookings**

```
SELECT status, COUNT(*) AS booking_count
FROM booking
GROUP BY status
ORDER BY booking_count DESC;
```



Result Grid | Filter Rows:

| status | booking_count |
|---|---|
| CONFIRMED | 3 |
| CANCELLED | 1 |

**Explanation**: Displays booking statuses in descending order of booking count.

**11. Use multiple aggregate functions together**

```
SELECT
   COUNT(*) AS total_shows,
   MIN(ticket_price) AS minimum_price,
   MAX(ticket_price) AS maximum_price,
   ROUND(AVG(ticket_price), 2) AS average_price,
   SUM(ticket_price) AS total_price
FROM showtime;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| total_shows | minimum_price | maximum_price | average_price | total_price |
|---|---|---|---|---|
| 5 | 180.00 | 300.00 | 230.00 | 1150.00 |

**Explanation:** Displays the total number of shows, minimum price, maximum price, average price, and sum of ticket prices in a single result row.

**B.2: Observation**

1. Aggregate functions were used to calculate customer counts, minimum and maximum ticket prices, average ticket prices, and total prices.
2. The COUNT() function was used to count customers and bookings.

3. GROUP BY was used to group records by city, booking status, and show ID.
4. The HAVING clause was used to filter groups based on aggregate conditions.
5. The WHERE clause was used to filter individual rows before grouping.
6. ORDER BY was used to arrange grouped results according to booking counts.

**Result:** Aggregate functions and grouping operations were successfully practised using MySQL Workbench.

## B.3: Answer the Following Questions

**1. What are aggregate functions? Explain COUNT(), SUM(), AVG(), MAX() and MIN() with suitable examples.**

Aggregate functions perform calculations on multiple rows and return a single result.

| Function | Purpose | Example |
|---|---|---|
| COUNT() | Counts rows or non-NULL values. | COUNT(*) |
| SUM() | Adds numeric values. | SUM(ticket_price) |
| AVG() | Calculates the average. | AVG(ticket_price) |
| MAX() | Returns the highest value. | MAX(ticket_price) |
| MIN() | Returns the lowest value. | MIN(ticket_price) |

Example:

```
    SELECT
  COUNT(*) AS total_shows,
  SUM(ticket_price) AS total_price,
  AVG(ticket_price) AS average_price,
  MAX(ticket_price) AS maximum_price,
  MIN(ticket_price) AS minimum_price
FROM showtime;
```

**2. Explain COUNT(*) and COUNT(DISTINCT column_name) with suitable examples**.

- `COUNT(*)` counts all rows.
- `COUNT(DISTINCT column_name)` counts distinct non-NULL values.

Examples:
SELECT COUNT(*)
FROM customer;

**3.How is COUNT() used with GROUP BY to count rows in each group? Explain with an example.**

GROUP BY divides records into groups based on a column, and COUNT() counts the rows within each group.

Example:

SELECT status, COUNT(*) AS booking_count

FROM booking

GROUP BY status;

**4. What is the purpose of the HAVING clause? Explain how COUNT() can be used with GROUP BY and HAVING to filter groups.**

The HAVING clause filters groups after grouping and aggregation. It is commonly used with aggregate functions.

Example :

```
SELECT status, COUNT(*) AS booking_count

FROM booking

GROUP BY status

HAVING COUNT(*) > 1;
```

**5.What is the difference between the WHERE and HAVING clauses in aggregate queries?**

| WHERE | HAVING |
|---|---|
| Filters individual rows. | Filters groups of rows. |
| Applied before grouping. | Applied after grouping. |
| Commonly uses row-level conditions. | Commonly uses aggregate conditions. |
| Example: WHERE status = 'CONFIRMED' | Example: HAVING COUNT(*) > 1 |